

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

Practical – I

Roll No.: T016	Name: Himani Malankar
Class:TYIT	Batch:1
Date of Assignment: 27/06/2026	Date/Time of Submission: 27/06/2026

AIM: INTRODUCTION TO AI AND INTELLIGENT AGENTS.

1. Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment and Autonomous Taxi. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

CODE:

```
table = {
    # Location, Status, Obstacle, Battery
    # Room A
    ('A', 'Dirty', 'Clear', 'High'): 'Suck',
    ('A', 'Clean', 'Clear', 'High'): 'Move_Right',
    ('A', 'Dirty', 'Obstacle', 'High'): 'Avoid_Obstacle -> Suck',
    ('A', 'Clean', 'Obstacle', 'High'): 'Avoid_Obstacle -> Move_Left',
    ('A', 'Clean', 'Clear', 'Low'): 'Go_To_Dock',

    # Room B
    ('B', 'Dirty', 'Clear', 'High'): 'Suck',
    ('B', 'Clean', 'Clear', 'High'): 'Move_Right',
    ('B', 'Dirty', 'Obstacle', 'High'): 'Avoid_Obstacle -> Suck',
    ('B', 'Clean', 'Obstacle', 'High'): 'Avoid_Obstacle -> Move_Left',
    ('B', 'Clean', 'Clear', 'Low'): 'Go_To_Dock',
}

sequence_table = {
    (('A', 'Dirty', 'Clear', 'High'), ('A', 'Clean', 'Clear', 'High')): 'Move_Right',
    (('A', 'Dirty', 'Clear', 'High'), ('B', 'Dirty', 'Clear', 'High')): 'Suck',
    (('A', 'Dirty', 'Clear', 'High'), ('B', 'Clean', 'Clear', 'High')): 'Move_Right',
    (('A', 'Dirty', 'Clear', 'High'), ('B', 'Dirty', 'Obstacle', 'High')): 'Avoid_Obstacle -> Suck',
    (('A', 'Dirty', 'Clear', 'High'), ('B', 'Clean', 'Obstacle', 'High')): 'Avoid_Obstacle ->
Move_Left',
    (('A', 'Clean', 'Clear', 'High'), ('A', 'Clean', 'Clear', 'Low')): 'Go_To_Dock',
    (('B', 'Dirty', 'Clear', 'High'), ('B', 'Clean', 'Clear', 'High')): 'Move_Right',
    (('B', 'Dirty', 'Clear', 'High'), ('B', 'Dirty', 'Obstacle', 'High')): 'Avoid_Obstacle -> Suck',

    (('B', 'Dirty', 'Clear', 'High'), ('B', 'Clean', 'Obstacle', 'High')): 'Avoid_Obstacle ->
Move_Left',

    (('B', 'Clean', 'Clear', 'High'), ('B', 'Clean', 'Clear', 'Low')): 'Go_To_Dock'
}
```

```

# Store percept history
percepts = []

# Table-driven agent
def table_driven_agent(percept):
    percepts.append(percept)

    # Check last two percepts in sequence table
    if len(percepts) >= 2:
        last_two = tuple(percepts[-2:])
        if last_two in sequence_table:
            return sequence_table[last_two]

    # Otherwise check simple lookup table
    return table.get(percept, "NoOperation")

# Main Program
print("==== Vacuum Cleaner Table-Driven Agent (2 Rooms) =====")

n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i+1}")

    location = input("Enter Location (A/B): ").upper()
    status = input("Enter Status (Clean/Dirty): ").capitalize()
    obstacle = input("Enter Obstacle (Clear/Obstacle): ").capitalize()
    battery = input("Enter Battery (High/Low): ").capitalize()

    percept = (location, status, obstacle, battery)

    action = table_driven_agent(percept)

    print("\nPercept History :", percepts)
    print("Agent Action   :", action)

print("\nProgram Executed Successfully.")
print("Himani Malankar")

```

OUTPUT:

```

==== Vacuum Cleaner Table-Driven Agent (2 Rooms) =====

Percept 1

Percept History : [('A', 'Dirty', 'Obstacle', 'High')]
Agent Action   : Avoid_Obstacle -> Suck

Percept 2

Percept History : [('A', 'Dirty', 'Obstacle', 'High'), ('A', 'Clean', 'Clear', 'High')]
Agent Action   : Move_Right

Percept 3

Percept History : [('A', 'Dirty', 'Obstacle', 'High'), ('A', 'Clean', 'Clear', 'High'), ('B', 'Dirty', 'Obstacle', 'High')]
Agent Action   : Avoid_Obstacle -> Suck

Percept 4

Percept History : [('A', 'Dirty', 'Obstacle', 'High'), ('A', 'Clean', 'Clear', 'High'), ('B', 'Dirty', 'Obstacle', 'High'), ('B', 'Clean', 'Clear', 'Low')]
Agent Action   : Go_To_Dock

Program Executed Successfully.
Himani Malankar

```

2. Design a Simple Rational Agent by defining PEAS for Autonomous Taxi Environment. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

CODE:

```
# (Location, Traffic Signal, Passenger in Car, Ride Request)
table = {
    ('A', 'Green', 'No', 'Yes'): 'Pick Up Passenger',
    ('A', 'Green', 'No', 'No'): 'Move Forward',
    ('B', 'Red', 'Yes', 'No'): 'Stop',
    ('B', 'Green', 'Yes', 'No'): 'Drive to Destination',
    ('C', 'Yellow', 'No', 'No'): 'Slow Down',
    ('C', 'Green', 'No', 'No'): 'Pick Up Passenger',
    ('D', 'Red', 'Yes', 'No'): 'Stop',
    ('D', 'Green', 'Yes', 'Yes'): 'Drop Passenger'
}
sequence_table = {
    (('A', 'Green', 'No', 'Yes'), ('A', 'Green', 'No', 'No')): 'Pick Up Passenger',
    (('A', 'Green', 'No', 'No'), ('B', 'Green', 'Yes', 'No')): 'Drive to Destination',
    (('B', 'Red', 'Yes', 'No'), ('B', 'Green', 'Yes', 'No')): 'Drive to Destination',
    (('B', 'Green', 'Yes', 'No'), ('D', 'Red', 'Yes', 'No')): 'Stop',
    (('D', 'Red', 'Yes', 'No'), ('D', 'Green', 'Yes', 'Yes')): 'Drop Passenger',
    (('C', 'Yellow', 'No', 'No'), ('C', 'Green', 'No', 'No')): 'Slow Down'
}

percepts = []

def taxi_agent(percept):
    percepts.append(percept)

    if len(percepts) >= 2:
        last_two = tuple(percepts[-2:])
        if last_two in sequence_table:
            return sequence_table[last_two]

    return table.get(percept, "Wait")

print("==== Autonomous Taxi Table-Driven Agent =====")
n = int(input("Enter the number of percepts: "))
for i in range(n):
    print(f"\nPercept {i + 1}")

    location = input("Location (A/B/C/D): ").upper()
    signal = input("Traffic Signal (Green/Yellow/Red): ").capitalize()
    passenger = input("Passenger in Car? (Yes/No): ").capitalize()
    request = input("Ride Request? (Yes/No): ").capitalize()
    percept = (location, signal, passenger, request)
    action = taxi_agent(percept)

    print("Percept History:", percepts)
    print("Current Percept:", percept)
    print("Agent Action:", action)
print("Himani T016")
```

OUTPUT:

===== Autonomous Taxi Table-Driven Agent =====

Percept 1

Percept History: [('A', 'Green', 'No', 'Yes')]

Current Percept: ('A', 'Green', 'No', 'Yes')

Agent Action: Pick Up Passenger

Percept 2

Percept History: [('A', 'Green', 'No', 'Yes'), ('B', 'Red', 'Yes', 'No')]

Current Percept: ('B', 'Red', 'Yes', 'No')

Agent Action: Stop

Percept 3

Percept History: [('A', 'Green', 'No', 'Yes'), ('B', 'Red', 'Yes', 'No'), ('B', 'Green', 'Yes', 'No')]

Current Percept: ('B', 'Green', 'Yes', 'No')

Agent Action: Drive to Destination

Percept 4

Percept History: [('A', 'Green', 'No', 'Yes'), ('B', 'Red', 'Yes', 'No'), ('B', 'Green', 'Yes', 'No'), ('D', 'Red', 'Yes', 'No')]

Current Percept: ('D', 'Red', 'Yes', 'No')

Agent Action: Stop

Percept 5

Percept History: [('A', 'Green', 'No', 'Yes'), ('B', 'Red', 'Yes', 'No'), ('B', 'Green', 'Yes', 'No'), ('D', 'Red', 'Yes', 'No'), ('D', 'Green', 'Yes', 'Yes')]

Current Percept: ('D', 'Green', 'Yes', 'Yes')

Agent Action: Drop Passenger

Himani T016

3. Design a Simple Rational Agent by defining PEAS. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

CODE:

```
# (Location, Temperature, Motion Detected)

table = {
    ('Living Room', 'Hot', 'Yes'): 'Turn On AC',
    ('Living Room', 'Cold', 'No'): 'Turn Off AC',

    ('Bedroom', 'Hot', 'Yes'): 'Turn On Fan',
    ('Bedroom', 'Cold', 'No'): 'Turn Off Fan',

    ('Kitchen', 'Hot', 'Yes'): 'Start Exhaust Fan',
    ('Kitchen', 'Cold', 'No'): 'Turn Off Exhaust Fan',

    ('Garage', 'Cold', 'Yes'): 'Turn On Light',
    ('Garage', 'Hot', 'No'): 'Turn Off Light'
}

percepts = []

def smart_home_agent(percept):
    percepts.append(percept)
    return table.get(percept, "No Action")

print("Smart Home Table-Driven Agent")

n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i + 1}")

    location = input("Enter Location (Living Room/Bedroom/Kitchen/Garage): ").title()
    temperature = input("Enter Temperature (Hot/Cold): ").capitalize()
    motion = input("Motion Detected? (Yes/No): ").capitalize()

    percept = (location, temperature, motion)

    action = smart_home_agent(percept)

    print("Current Percept :", percept)
    print("Percept History :", percepts)
    print("Agent Action  :", action)
```

OUTPUT:

Smart Home Table-Driven Agent

Percept 1

Current Percept : ('Living Room', 'Hot', 'Yes')

Percept History : [('Living Room', 'Hot', 'Yes')]

Agent Action : Turn On AC

Percept 2

Current Percept : ('Bedroom', 'Cold', 'No')

Percept History : [('Living Room', 'Hot', 'Yes'), ('Bedroom', 'Cold', 'No')]

Agent Action : Turn Off Fan

Percept 3

Current Percept : ('Kitchen', 'Hot', 'Yes')

Percept History : [('Living Room', 'Hot', 'Yes'), ('Bedroom', 'Cold', 'No'), ('Kitchen', 'Hot', 'Yes')]

Agent Action : Start Exhaust Fan

Percept 4

Current Percept : ('Garage', 'Cold', 'No')

Percept History : [('Living Room', 'Hot', 'Yes'), ('Bedroom', 'Cold', 'No'), ('Kitchen', 'Hot', 'Yes'), ('Garage', 'Cold', 'No')]

Agent Action : No Action